

WEEK 1 - SESSION C1

Web Architecture: Client/Server, DNS, HTTP, Status Codes, Rendering & Hosting

Student Learning Material

The big question for this session

What actually happens after you type a URL such as `https://example.com`, press Enter, and see a web page?

This material is designed to give you a working mental model of the modern web. You do not need to become a network engineer to become a good front-end developer. However, you must understand what your browser, the network, the server, HTTP, DNS, rendering engine, and hosting platform are doing around your code.

Learning Objectives

- Explain the roles of a client and a server in a web application.
- Describe how a URL is interpreted and how DNS helps a browser find a server.
- Explain an HTTP request and HTTP response, including methods, headers, body, and status codes.
- Recognize the meaning of the 2xx, 3xx, 4xx, and 5xx status-code families.
- Describe the basic browser rendering pipeline from HTML to a visible page.
- Distinguish local development, a server, a domain, and hosting/deployment.
- Use browser DevTools to inspect a real network request.
- Trace the complete journey from URL entry to the final rendered page.

What you should be able to say by the end

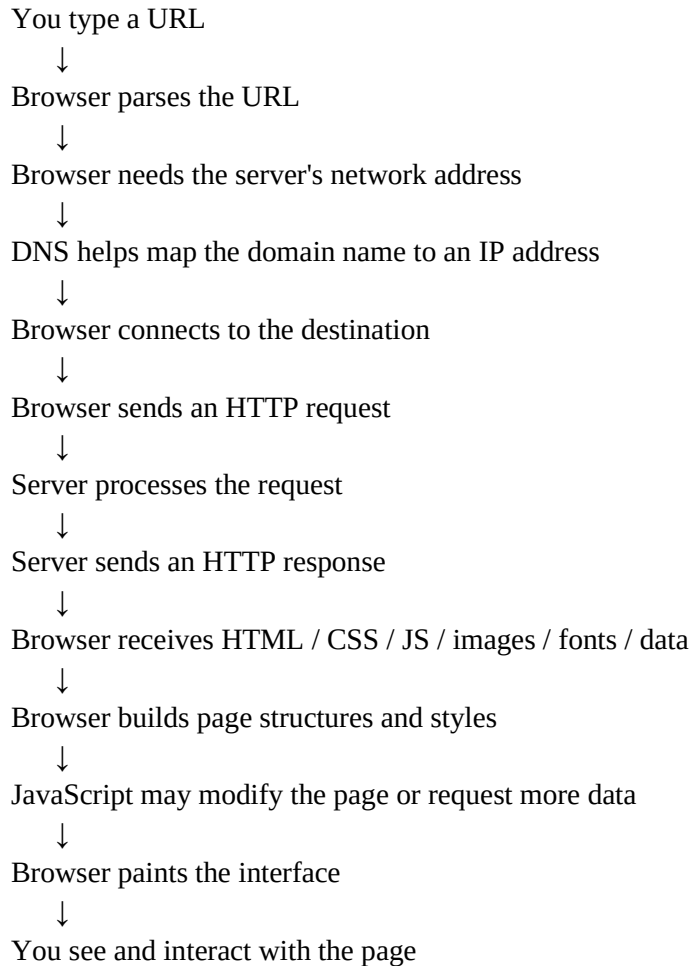
Target answer

When I enter a URL, the browser parses it, resolves the domain through DNS when necessary, establishes communication with the destination server, sends an HTTP request, receives an HTTP response, downloads the required resources, builds the page structures and styles, executes JavaScript when needed, and paints the resulting interface. Hosting makes the application/resources available from an Internet-accessible server.

1. Start With the Journey: What Happens After Enter?

Before memorizing terms, build one mental model. Imagine that you type `https://example.com/products` into the browser and press Enter. The browser does not magically 'open a website'. It performs a sequence of operations that eventually produces pixels on your screen.

A simplified journey



Important

This is a simplified teaching model. Real browsers use caching, connection reuse, HTTP/2 or HTTP/3, security checks, service workers, compression, preloading and many other optimizations. Learn the core flow first.

2. Client and Server: The Two-Sided Model

A web application usually involves at least two roles: a client and a server. The client is normally the browser or another application making a request. The server is the system that receives requests and provides resources or performs work.

Client	Server
Usually the browser/app used by the user	A computer/process that receives requests
Sends requests	Receives and processes requests
Renders the user interface	May generate HTML or return data
Runs front-end JavaScript	Runs server-side code when applicable
Stores some local state/cache	Stores or accesses persistent data
Displays the response	Sends a response

Think of it like a restaurant

- Client = customer placing an order.
- Request = the order.
- Server = kitchen/service system preparing the order.
- Response = the prepared order.
- Database = storage from which the kitchen may obtain ingredients/information.
- Browser UI = what the customer ultimately sees.

The analogy is useful but imperfect. A server is not necessarily a single physical machine, and a web application may involve many services, caches, databases, CDNs and APIs.

Frontend vs backend

Frontend code is primarily concerned with what runs in or directly supports the user's interface: layout, components, interactions, browser state and communication with APIs. Backend code handles server-side responsibilities such as authentication logic, business rules, database access and API endpoints.

Do not confuse 'server' with 'backend code'

A server is the environment/process that responds to requests. Backend is the broader set of server-side application responsibilities. A static hosting server can return files without running a traditional backend application.

3. What Is a URL?

URL stands for Uniform Resource Locator. It tells the browser where a resource can be found and, through its structure, provides information about how to access it.

<https://www.example.com:443/products?category=shoes&sort=price#sale>

Part	Example	Purpose
------	---------	---------

Scheme / protocol	https	Defines the communication scheme.
Host / domain	www.example.com	Human-readable address of the destination.
Port	443	Logical network endpoint; often omitted because HTTPS commonly uses 443.
Path	/products	Identifies a resource or route on the host.
Query string	?category=shoes&sort=price	Provides parameters to the requested resource.
Fragment	#sale	Identifies a location/state within the client-side document.

HTTP vs HTTPS

HTTP is the application-level protocol used for web communication. HTTPS is HTTP protected by TLS. For a front-end developer, the practical point is that HTTPS helps protect data in transit and provides server authentication through certificates. You do not need to study the cryptographic mathematics in this session.

4. DNS: Turning a Name Into a Network Destination

Humans prefer names such as example.com. Networks communicate using IP addresses such as IPv4 or IPv6 addresses. DNS, or Domain Name System, provides the naming system that lets software discover the address information associated with a domain.

You type:

https://example.com

Conceptually:

example.com

↓ DNS lookup

IP address information

↓

browser can contact the destination

Why do we need DNS?

- People can remember names more easily than numeric network addresses.
- A domain can be moved to a different server without changing the name users type.
- DNS supports a distributed naming infrastructure for the Internet.
- Different DNS record types support different purposes, such as web destinations and mail services.

A useful simplified DNS sequence

1. The browser checks whether it already knows the answer from cache or an existing connection/context.
2. The operating system or configured resolver may be consulted.
3. A DNS resolver obtains the required DNS information if it is not already cached.
4. The browser receives an address and can attempt to contact the destination.

5. DNS information has a lifetime controlled by caching rules such as TTL.

Do not say: 'DNS is the Internet.'

DNS is a naming system. It helps map names to network information. It does not itself deliver the web page.

5. HTTP: The Language of Web Requests and Responses

HTTP (Hypertext Transfer Protocol) defines how a client and server communicate application-level requests and responses. As a front-end developer, HTTP is one of the most important concepts in this course because browser applications constantly communicate with servers.

An HTTP request

GET /products?category=shoes HTTP/1.1

Host: example.com

Accept: text/html

User-Agent: Browser

A request can be understood as having a method, a target/URL, headers, and sometimes a body. The exact wire representation depends on the HTTP version, but the conceptual pieces remain useful.

Component	Meaning	Example
Method	What kind of operation the client is asking for	GET, POST, PUT, PATCH, DELETE
URL/target	Where the request is directed	/products/42
Headers	Metadata about the request	Accept, Content-Type, Authorization
Body	Data sent with the request when applicable	JSON form data

Common HTTP methods

Method	Typical purpose	Example
GET	Retrieve data/resource	GET /api/products
POST	Submit data or request creation/action	POST /api/products
PUT	Replace/update a resource	PUT /api/products/42
PATCH	Partially update a resource	PATCH /api/products/42
DELETE	Remove a resource	DELETE /api/products/42

Frontend rule of thumb

When you call `fetch('/api/products')`, your JavaScript is creating an HTTP request. The API does not 'live inside fetch'; fetch is simply a browser API that lets your code make the request.

An HTTP response

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 123

```
{"id":42,"name":"Running Shoes"}
```

Component	Meaning
Status code	Numeric result of the request, such as 200 or 404
Status text	Human-readable description in some HTTP representations
Response headers	Metadata about the returned response
Response body	The actual returned content, when present

6. HTTP Status Codes: Learn the Families First

Do not try to memorize every status code. First learn what the first digit tells you.

Family	Meaning	Mental model
1xx	Informational	The communication is providing information/progress.
2xx	Success	The request was successfully handled.
3xx	Redirection	More action/location information is involved.
4xx	Client-side request problem	The request cannot be fulfilled as sent.
5xx	Server-side failure	The server failed while handling a valid-looking request.

Important codes for frontend developers

Code	Meaning	Typical frontend interpretation
200	OK	Request succeeded.
201	Created	A resource was successfully created.
204	No Content	Success, but there is no

		response body to consume.
301 / 308	Permanent redirect	Resource has a new permanent location.
302 / 307	Temporary redirect	Resource is temporarily available elsewhere.
400	Bad Request	Server cannot process the request as sent.
401	Unauthorized	Authentication is required or invalid.
403	Forbidden	Server understood the request but refuses access.
404	Not Found	Requested resource/route was not found.
405	Method Not Allowed	Endpoint exists but does not support that method.
409	Conflict	Request conflicts with current resource state.
422	Unprocessable Content	Input is understood but fails application validation in common API usage.
429	Too Many Requests	Rate limit has been exceeded.
500	Internal Server Error	Unexpected server-side failure.
502	Bad Gateway	A gateway/proxy received an invalid upstream response.
503	Service Unavailable	Service is temporarily unable to handle the request.

Critical distinction

404 and 500 are not the same problem. A 404 generally means the requested resource/route was not found. A 500 means the server encountered an unexpected problem while handling the request.

7. Headers: Metadata That Controls Communication

Headers are key-value metadata attached to requests and responses. You will encounter them constantly in DevTools, APIs, authentication, caching, content negotiation and debugging.

Header	Where	Why it matters
Content-Type	Request/response	Describes the format of the body, e.g. application/json.
Accept	Request	Indicates which response formats the client can handle.
Authorization	Request	Carries authentication credentials/token information

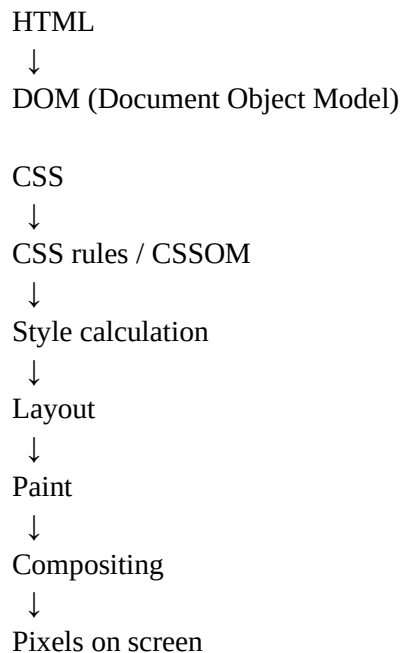
		when used.
Cache-Control	Usually response	Provides caching instructions.
Location	Response	Often identifies the target of a redirect.
Set-Cookie	Response	Asks the browser to store cookie information.
Cookie	Request	Sends applicable cookies back to the server.
Content-Length	Request/response	Describes body length in relevant HTTP representations.
User-Agent	Request	Provides client software information.

Security reminder

Never expose secrets merely because a header contains a token. Authentication tokens and API keys must be handled according to the security design of the application. Do not paste real credentials into screenshots, assignments or public repositories.

8. Rendering: How Code Becomes What You See

A browser does much more than download HTML. It parses documents, constructs internal structures, applies CSS, executes JavaScript, calculates layout, and paints pixels to the screen.



JavaScript can interact with the DOM/CSS and trigger changes along the way.

HTML → DOM

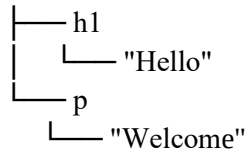
The browser parses HTML and constructs a DOM tree. The DOM is a programming representation of the document. JavaScript can query and modify it.

HTML:

```
<h1>Hello</h1>  
<p>Welcome</p>
```

Conceptual DOM:

Document



CSS → Styles

CSS rules tell the browser how elements should look and participate in layout: dimensions, colors, spacing, typography, positioning, grids, flexbox and more.

JavaScript → Behavior

JavaScript can respond to user events, modify the DOM, change styles, manage application state, communicate with APIs and create dynamic interfaces.

A simplified rendering pipeline

1. Parse HTML and construct the DOM.
2. Parse relevant CSS and determine styles.
3. Calculate layout: where and how large elements should be.
4. Paint visual information such as text, backgrounds, borders and images.
5. Composite layers and display the final result.

Frontend connection

When you build React applications, React does not replace the browser's rendering engine. React helps you describe and manage UI updates; the browser still ultimately renders HTML/CSS-related structures and paints pixels.

9. Static Pages, Dynamic Pages and APIs

A web page can be as simple as a set of files or as complex as a distributed application. Understanding the difference helps you choose appropriate tools.

Type	What happens	Example
Static resource	Server returns a stored file/resource.	HTML, CSS, JS, image
Server-rendered page	Server executes logic and produces HTML for a request.	Template-based web app
Client-rendered application	Browser loads JS and constructs/updates much of the UI.	Many React SPA patterns
API-driven UI	Frontend requests data from an API and renders it.	React + REST/JSON API
Hybrid	Server and client both contribute to rendering and behavior.	Modern frameworks such as Next.js

These categories can overlap. A modern framework may pre-render some content, dynamically render other content, and then hydrate or enhance the page in the browser.

10. Local Development vs Hosting vs Deployment

One of the most common beginner confusions is thinking that a project running on localhost is already 'online'. It is not. Local development means the application is being served from your own machine or local environment.

Term	Meaning
localhost	A name commonly used to refer to your own computer.
Local development server	A program that serves your project while you develop it.
Build	The process of transforming source code into a deployable output when required.
Deployment	Putting an application/version into a target environment.
Hosting	Providing infrastructure/service that makes application resources available.
Domain	A human-friendly Internet name used to reach a service.
Production	The environment intended for real users.
CDN	A distributed network that can serve content closer to users.

Development:

your computer
↓
http://localhost:3000
↓
you

Production:

user browser
↓
domain
↓
DNS
↓
hosting/CDN/server
↓
application/resources

Simple distinction

Development answers: 'Where am I building and testing this?' Hosting answers: 'Where is the application/resources being served from so other users can access them?'

11. Guided Practice: Use DevTools Like a Frontend Developer

The Network panel is one of your most important debugging tools. It lets you see what the browser requested, what came back, how long it took, and what metadata was exchanged.

Practice procedure

1. Open a modern browser such as Chrome or Firefox.
2. Open Developer Tools (usually F12 or Ctrl+Shift+I).
3. Select the Network tab.
4. Reload the page so you can observe the requests from the beginning.
5. Click a document request, usually the main HTML document.
6. Identify the request method.
7. Identify the requested URL.
8. Find the status code.
9. Inspect request and response headers.
10. Look for the response or preview when the browser exposes it.
11. Observe timing information.
12. Open a resource such as CSS, JavaScript or an image and compare it with the document request.

What to look for

Question	Where to investigate
What URL was requested?	Request URL
Was it GET or POST?	Request Method
Did it succeed?	Status Code
What type of content came back?	Content-Type response header
Was the browser redirected?	3xx status / redirect information
Was the resource missing?	4xx, often 404
Did the server fail?	5xx
How long did it take?	Timing
What data was sent?	Request Payload, when applicable
What data came back?	Response/Preview

12. Full Worked Example: Visiting <https://example.com>

The following is a conceptual trace. Real browser behavior can skip or reorder work because of caching, connection reuse and other optimizations.

Stage	What it means
1. URL entry	The browser parses the scheme, host, path and other URL components.
2. Cache checks	The browser may already have relevant DNS information, connections or resources.
3. DNS	If needed, the browser obtains network address information for the host.
4. Connection	The browser establishes or reuses a suitable network connection to the destination.
5. Security	For HTTPS, TLS provides an encrypted and authenticated channel.
6. HTTP request	The browser sends a request for the resource.
7. Server processing	The server, application, CDN or related infrastructure determines how to respond.
8. HTTP response	The browser receives a status code, headers and usually a body.
9. Parse HTML	The browser begins constructing the DOM and discovers referenced resources.
10. Fetch dependencies	CSS, JavaScript, images, fonts and other resources may be requested.
11. Style/layout	The browser computes styles and layout.
12. JavaScript	Scripts may execute, respond to events, update UI and request data.
13. Paint/composite	The browser produces the visual result.
14. Interaction	Your next click, scroll or input may trigger additional events and network requests.

13. Debugging With Status Codes: Think Like an Investigator

When a page or API call fails, do not immediately change code randomly. Start by identifying where the failure occurs.

Symptom	Possible interpretation	First thing to inspect
404 for API endpoint	Wrong URL/path or route does not exist	Request URL + backend route
401	Authentication is missing/invalid	Auth state + Authorization/cookie
403	Access is denied	Permissions/auth rules
400	Request format/data may be invalid	Payload + request headers
500	Backend encountered an error	Server logs + request details
Slow request	Network/server/resource issue	Timing + response size + server
Page appears unstyled	CSS failed to load or apply	CSS request/status + console
JS UI not working	Script error or logic issue	Console + JS request/status
Wrong data displayed	API/data/state issue	Response body + application state

Debugging principle

Observe first, hypothesize second, change third. DevTools gives you evidence. Use that evidence before editing code.

14. Console vs Network: Know the Difference

Tool	Best for
Console	JavaScript errors, warnings, logs, runtime behavior
Network	HTTP requests, responses, status codes, headers, payloads, timing
Elements/Inspector	DOM structure, applied styles and layout inspection
Sources	Inspecting/debugging loaded source and scripts
Application/Storage	Cookies, local storage, session storage and related browser data

15. Why This Matters for Your Frontend Career

A frontend developer is not only someone who can write JSX, CSS or JavaScript. A strong frontend developer understands the environment in which that code runs.

- If an API returns 401, you should understand that authentication is involved rather than assuming React is broken.
- If an asset returns 404, you should inspect the requested path and deployment configuration.

- If a page is slow, you should be able to inspect network timing and resource sizes.
- If a button appears but does nothing, you should distinguish a UI/rendering issue from a JavaScript/runtime issue.
- If an application works on localhost but fails after deployment, you should consider environment variables, routes, asset paths, hosting configuration, HTTPS, API URLs and backend accessibility.
- If a page is visible before JavaScript finishes, you should understand that browsers can render HTML/CSS independently of your application's later behavior.

16. Concepts You Must Not Mix Up

Do not confuse	Correct distinction
URL and IP address	URL is a resource locator; IP is a network address.
DNS and HTTP	DNS helps find address information; HTTP communicates application requests/responses.
Browser and server	Browser is commonly the client; server provides/handles resources or requests.
Frontend and frontend framework	Frontend is the user-facing application layer; React is one tool/framework/library used to build it.
localhost and Internet hosting	localhost normally refers to your own machine; hosting makes resources available from an external environment.
404 and 500	404 is generally not found; 500 indicates an internal server failure.
HTTP and HTTPS	HTTPS is HTTP over a TLS-protected connection.
HTML and DOM	HTML is markup; DOM is the browser's in-memory document representation.
CSS and visual pixels	CSS provides rules; the browser calculates layout/paint to produce pixels.
JavaScript and the browser	JavaScript is a language; the browser provides APIs/runtime capabilities in which it executes.

17. In-Class Questions and Activities

Activity A - Explain the journey

Without looking at the material, explain what happens from entering a URL until the page becomes visible. Your explanation should include:

- URL parsing
- DNS

- connection
- HTTP request
- HTTP response
- status code
- HTML/CSS/JavaScript
- rendering

Activity B - Status-code detective

For each situation, choose the most likely status family/code and explain why.

Situation	Your answer
<i>A page is successfully returned.</i>	
<i>A user tries to access a protected resource without valid authentication.</i>	
<i>A requested route does not exist.</i>	
<i>The server crashes while processing the request.</i>	
<i>The requested resource has been moved elsewhere.</i>	
<i>The user sends malformed/invalid request data.</i>	

Activity C - Network investigation

Open DevTools on a public site or a site you are developing. Pick one document request and one API/data request. Record the method, URL, status code, content type, and approximate timing. Then explain the difference between the two.

18. Homework / Self-Practice

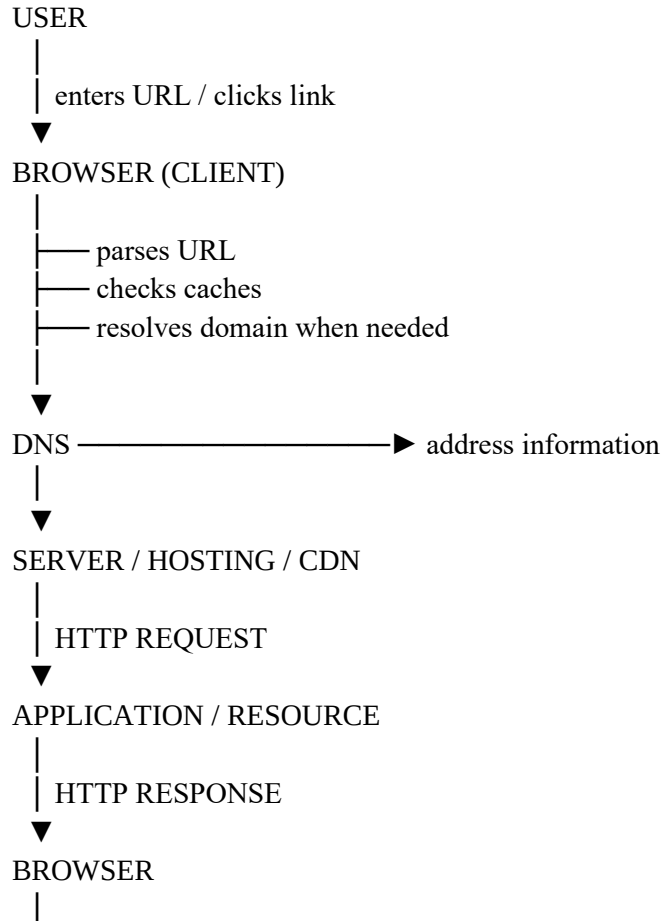
1. Draw the complete journey of a browser request from URL entry to rendered page.
2. Take any public website and inspect at least five Network requests.
3. For three requests, record the method, status code, response content type and timing.
4. Find one request that returns a 2xx response and one that returns a 3xx/4xx response if possible.
5. Explain the difference between DNS and HTTP in your own words.
6. Explain why a project can work at localhost but fail after deployment.
7. Write a short explanation of what the browser does with HTML, CSS and JavaScript.

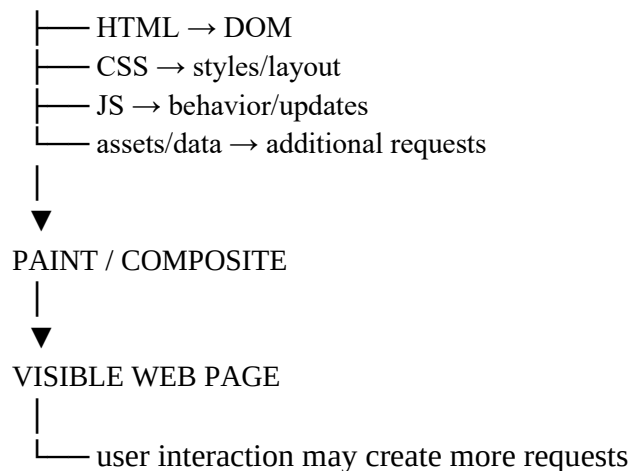
20. Essential Glossary

Term	Student-friendly definition
API	A defined interface through which software can communicate with another system.
Browser	A client application that retrieves, interprets and displays web content.
CDN	A distributed network used to deliver content efficiently from locations closer to users.
Client	The software that initiates a request, commonly a browser.

DNS	The Internet's distributed naming system used to discover address information for domain names.
Domain	A human-readable name such as example.com.
HTTP	A protocol defining web request/response communication.
HTTPS	HTTP protected using TLS.
Hosting	Providing infrastructure/service where web resources or applications are served.
IP address	A network address used to identify an interface/destination in an IP network.
localhost	A conventional hostname referring to the local machine.
Request	A message sent by a client asking for a resource or operation.
Response	A server-side reply to a request.
Status code	A numeric code describing the result/category of an HTTP request.
DOM	The browser's object representation of an HTML document.
Rendering	The browser process of turning document/style information into a visual result.
Deployment	Making a particular application version available in a target environment.
Production	The environment intended for real users.
Header	Metadata attached to an HTTP request or response.
Payload/body	The data carried in the body of an HTTP message.

21. The One Mental Model to Keep





If you remember only one thing

The web is a conversation between software systems. The browser requests resources or actions, servers and related infrastructure respond, and the browser turns the received information into an interactive experience.

Final self-check

- I can explain client and server without memorizing a textbook definition.
- I can break a URL into its important parts.
- I can explain why DNS exists.
- I can describe the major parts of an HTTP request and response.
- I can interpret the 2xx, 3xx, 4xx and 5xx families.
- I can explain what HTML, CSS and JavaScript contribute to rendering.
- I understand the difference between localhost and production hosting.
- I can inspect a request using the Network tab in DevTools.
- I can investigate a frontend problem using evidence instead of guessing.

22. Further Learning Direction

After this session, you will gradually build deeper knowledge of browser networking, APIs, authentication, cookies, storage, caching, performance, accessibility, security and deployment. Do not try to master all of these today. The goal of Session C1 is to give you a strong map of the territory so that later topics have a place to fit.